Lab: Creating Clusters

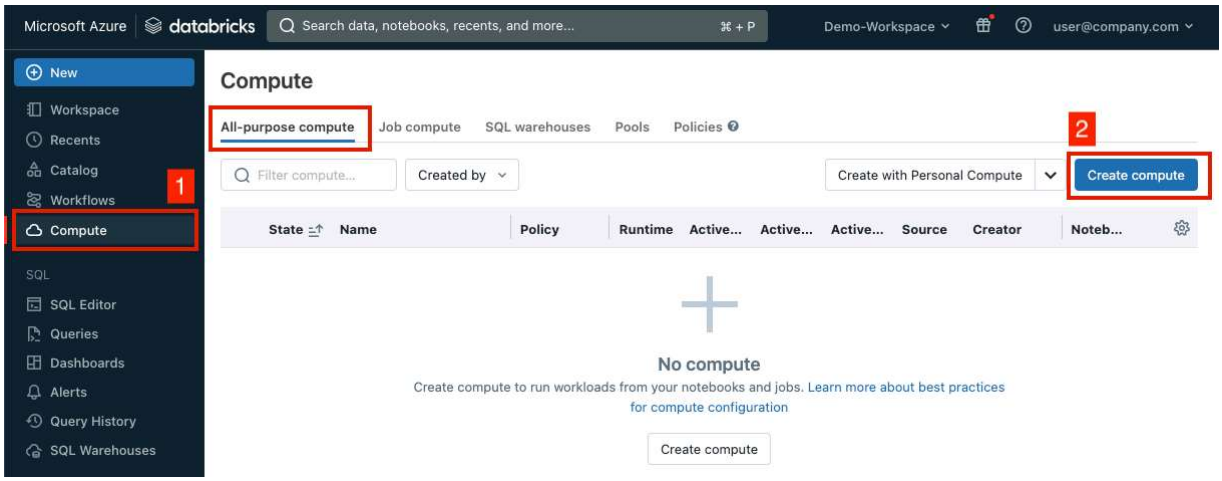
Q1 - Creating a Demo Cluster

Create a cluster with the following configurations:

Setting	Instructions
Cluster name	**Demo Cluster**
Cluster mode	**Signle node**
Runtime version	Select the Databricks runtime version 13.3 LTS
Photon Acceleration	Uncheck the option
Node type	4 cores
Auto termination	30 minutes

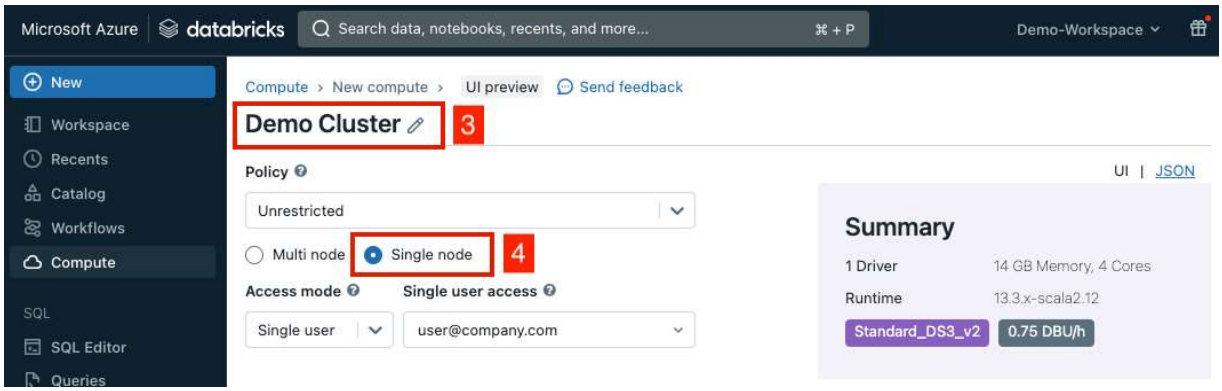
1- Navigate to the **Compute** tab in the left side bar.

2- Under **All-purpose compute** tab, click **Create compute**.



3- On top, click on the default name to change it. Name your cluster as **Demo Cluster**

4- Select **Single node** cluster



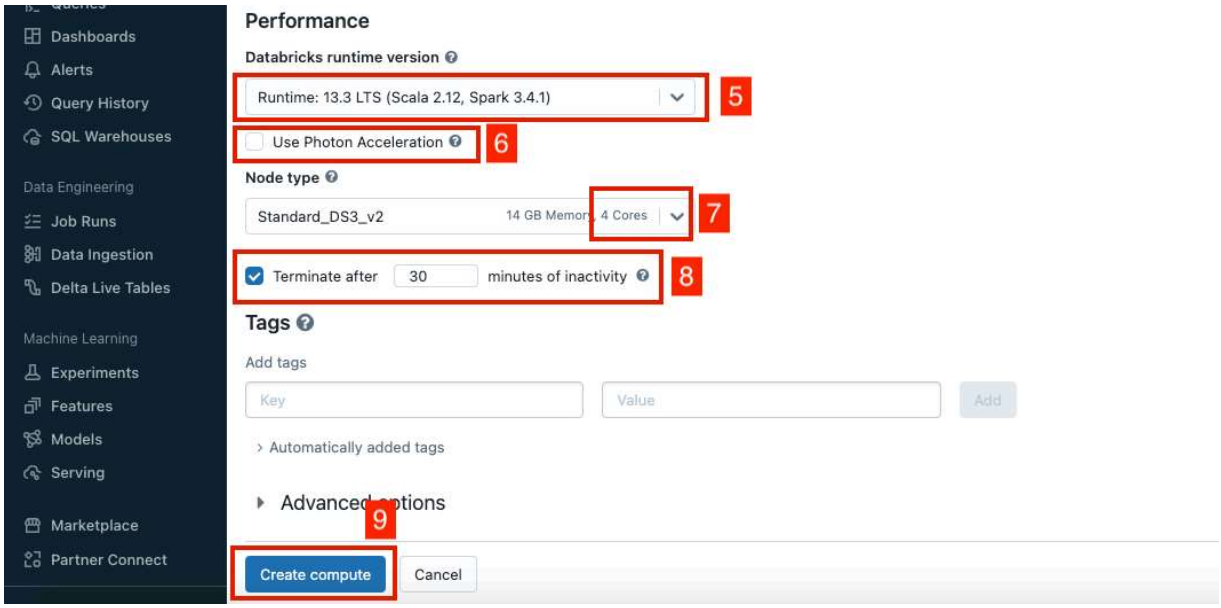
5- Select the Databricks runtime version 13.3 LTS (Long Term Support)

6- Uncheck the option for the **Use Photon Acceleration**

7- Select a Node type of 4 cores

8- Set the auto termination of the cluster to 30 minutes

9- Lastly, click **Create compute**.


Dashboards
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
Delta Live Tables
Machine Learning
Experiments
Features
Models
Serving
Marketplace
Partner Connect
Performance
Databricks runtime version
Runtime: 13.3 LTS (Scala 2.12, Spark 3.4.1)
5
Use Photon Acceleration
6
Node type
Standard_DS3_v2
14 GB Memory, 4 Cores
7
Terminate after
30
minutes of inactivity
8
Tags
Add tags
Key
Value
Add
Automatically added tags
Advanced options
9
Create compute
Cancel

(https://databricks.com)

Lab Solution: Get started with Databricks Notebook

Q1 - Renaming the Notebook

Change the name of the current notebook to "1.1L - My first lab"

Anwser: to change the name of the notebook, click on the name at the top of this page, then make changes to the name

Q2 - Attaching a cluster

Attach the cluster you created previously to this notebook

Anwser: to attach a cluster to this notebook, click the dropdown near the top-right corner of this page. Select your cluster.

Q3 - Execute a Python code

In the below cell, fill in the blank to print the result of adding the two variables x and y

```
6
```

```
x = 5
y = 10
result = x + y

# Answer
print(result)
```

Q4 - Execute a SQL cell

Change the language in the below cell to execute the SQL statement

```
8
```

```
%sql
SELECT 5 + 10
```

Q5 - Create a Markdown Cell

1. Insert a new cell below this one
2. In the new cell, add Markdown with a header and bullet points as shown in the following image



Animals

- Cats
- Dogs
- Birds

Q6 - Using %run command

1. Create a new Python notebook named **helper** in the current directory (i.e., in the **labs/1- Databricks Lakehouse Platform** folder)
2. In the **helper** notebook, declare a variable called **my_country** and assign it a value of your country name. For example:

```
my_country = "France"
```

3. In the following cell, execute a %run command to include the **helper** notebook into this current notebook

Hint: use a dot (**.**) to refer to the current directory

12

```
%run ./helper
```

Now, run the following cell to test that you are able to print the **my_country** variable

14

```
print(my_country)
```

Q7 - Functions definition

1. In the **helper** notebook, define a Python function named **addition()** that print the sum of two numbers
2. Execute again the above %run command to take the function definition into account
3. Run the below cell to call the function

16

```
num1 = 20
num2 = 30

addition(num1, num2)
```

17

Lab Solution: Delta Lake

Creating a Delta Table

Run the cell below to create the **persons** Delta Table, and apply some operations on it.

4

```
USE CATALOG hive_metastore;

CREATE OR REPLACE TABLE persons
  (id INT, name STRING, age INT);

INSERT INTO persons
VALUES
  (1, "Tom", 18),
  (2, "Kumar", 25);

INSERT INTO persons
VALUES
  (3, "Ali", 50),
  (4, "Sandra", 35);

INSERT INTO persons
VALUES
  (5, "Eric", 28),
  (6, "Salma", 42);

UPDATE persons
SET age = age + 10
WHERE id = 1;

DELETE FROM persons
WHERE name = "Eric";
```

Q1 - Querying Delta Lake table

Query the data in the **persons** table using **SELECT** statement

6

```
-- Answer
SELECT * FROM persons
```

Q2 - Checking table history

Review the history of the table transactions using the **DESCRIBE HISTORY** command

8

```
-- Answer
DESCRIBE HISTORY persons
```

Q3- Checking table metadata

Review the basic metadata information of the table using the **DESCRIBE DETAIL** command

```
10
--Answer
DESCRIBE DETAIL persons
```

Q4- Exploring table directory

Explore the table directory using the **%fs** magic command.

Hint: get the table location from the above metadata information

```
12
%fs ls 'dbfs:/user/hive/warehouse/persons'
```

Q5- Exploring the transactions log

Explore the **_delta_log** subfolder in the table directory

```
14
%fs ls 'dbfs:/user/hive/warehouse/persons/_delta_log'
```

```
15
```

Lab Solution: Databases and Tables on Databricks

Setting the default catalog

Run the cell below to set the current catalog to **hive_metastore**

```
4
USE CATALOG hive_metastore
```

Q1 - Creating managed table

In the default database, create a managed table named **movies_managed** that has the following schema:

Column Name	Column Type
title	STRING
category	STRING
length	FLOAT
release_date	DATE

```
6
-- Answer
CREATE TABLE movies_managed
  (title STRING, category STRING, length INT, release_date DATE);
```

Review the extended metadata information of the table, and verify that:

1. The table type is Managed
2. The table is located under the default hive directory

```
8
DESCRIBE EXTENDED movies_managed
```

Q2 - Creating external table

In the default database, create an external Delta table named **actors_external**, and located under the directory: **dbfs:/mnt/demo/actors_external**

The schema for the table:

Column Name	Column Type
actor_id	INT
name	STRING

```
10
-- Answer
CREATE OR REPLACE TABLE actors_external
  (actor_id INT, name STRING, nationality STRING)
LOCATION 'dbfs:/mnt/demo/actors_external';
```

Q4- Checking table metadata

Review the extended metadata information of the table, and verify that:

1. The table type is External
2. The table is located under the directory: **dbfs:/mnt/demo/actors_external**

```
12
DESCRIBE EXTENDED actors_external
```

Q3- Dropping manged table

Drop the manged table **movies_managed**

```
14
-- Answer
DROP TABLE movies_managed
```

Check that the directory of the managed table has been deleted

```
16
%fs ls 'dbfs:/user/hive/warehouse/movies_managed'
```

Q4- Drop external table

Drop the external table **actors_external**

```
18
DROP TABLE actors_external
```

Check that the directory of the external table has **Not** been deleted

```
20
%fs ls 'dbfs:/mnt/demo/actors_external'
```

Q5- Creating new schema

Create a new schema named **db_cinema**

```
22
```

```
-- Answer
CREATE SCHEMA db_cinema
```

Review the extended metadata information of the database, and verify that the database is located under the default hive directory.

Note that the database folder has the extenstion **.db**

```
24
DESCRIBE DATABASE EXTENDED db_cinema
```

Use the new schema to create the below **movies** table

```
26
-- Answer
USE db_cinema;

CREATE TABLE movies
  (title STRING, category STRING, length INT, release_date DATE);
```

Q6- Creating new schema in custom location

Create a new schema named **cinema_custom** in the directory: **dbfs:/Shared/schemas/cinema_custom.db**

```
28
-- Answer
CREATE SCHEMA cinema_custom
LOCATION 'dbfs:/Shared/schemas/cinema_custom.db'
```

Use the new schema to create the below **movies** table

```
30
USE cinema_custom;

CREATE TABLE movies
  (title STRING, category STRING, length INT, release_date DATE);
```

Finally, review the extended metadata information of the table **movies**, and verify that:

1. The table type is Managed
2. The table is located in the new database defined under the custom location

```
32
DESCRIBE EXTENDED movies
```

```
33
```

(https://databricks.com)

Lab Solution: Querying Files

Run the following cell to setup the lab environment

```
4
%run ../Includes/Setup-Lab
```

Q1- Extracting data directly from Parquet files

Use a SELECT statement to directly query the content of the Parquet files in the directory **${dataset.school}/enrollments**

```
6
-- ANSWER
SELECT * FROM parquet.`${dataset.school}/enrollments`
```

Use the above SELECT query in a CTAS statement to create the table **enrollments**

```
8
-- ANSWER
CREATE TABLE enrollments AS
SELECT * FROM parquet.`${dataset.school}/enrollments`
```

Run the below cell to ensure data was written as expected in the **enrollments** table

```
10
SELECT * FROM enrollments
```

Q2- Registering Tables from JSON Files

Use CTAS statement to create the table **students** from the json files in the directory: **${dataset.school}/students-json**

```
12
-- ANSWER
CREATE TABLE students AS
SELECT * FROM json.`${dataset.school}/students-json`;
```

Run the below cell to ensure data was written as expected in the **students** table

```
14
SELECT * FROM students
```

Q3- Registering Tables from CSV Files

Create the temporary view **courses_tmp_vw** from the csv files in the directory: **${dataset.school}/courses-csv**

Knowing that:
- The delimiter is semicolon (**;**)
- There is a header of column names in each file

The schema for the view:

Column Name	Column Type
course_id	STRING
title	STRING
instructor	STRING
category	STRING
price	DOUBLE

16

```
-- ANSWER
CREATE TEMP VIEW courses_tmp_vw
   (course_id STRING, title STRING, instructor STRING, category STRING, price DOUBLE)
USING CSV
OPTIONS (
  path = "${dataset.school}/courses-csv",
  header = "true",
  delimiter = ";"
);
```

Create the manged table **courses** from the temporary view **courses_tmp_vw**

18

```
-- ANSWER
CREATE TABLE courses AS
  SELECT * FROM courses_tmp_vw;
```

Query the data in the **courses** table to ensure data was written as expected.

20

```
-- ANSWER
SELECT * FROM courses
```

Finally, review the metadata information of the table **courses**, and verify that the table type is Managed

22

```
DESCRIBE EXTENDED courses
```

23

Lab Solution: Advanced ETL

Run the following cell to setup the lab environment

4

```
%run ../Includes/Setup-Lab
```

Q1- Interacting with JSON data

Review the nested data structures of the **profile** column in the **students** table created in the previous lab

6

```
SELECT email, profile
FROM students
```

7

```
DESCRIBE students
```

Use the appropriate syntax to access the **last_name** and **city** information from the **profile** column

9

```
-- ANSWER
SELECT email, profile:last_name AS student_surname, profile:address:city AS student_city
FROM students
```

Q2- Higher Order Functions

Review the array column **courses** in the **enrollments** table created in the previous lab

11

```
SELECT enroll_id, courses
FROM enrollments
```

Filter this array column to keep only course elements having subtotal greater than 40

13

```
-- ANSWER
SELECT
  enroll_id,
  courses,
  FILTER (courses, i -> i.subtotal > 40) AS large_totals
FROM enrollments
```

Q3- SQL UDFs

Define a UDF function named **get_letter_grade** that takes one parameter named **gpa** of type DOUBLE. It returns the corresponding letter grade as indicated in the following table:

GPA (4.0 Scale)	Grade Letter
3.50 - 4.0	A
2.75 - 3.44	B
2.0 - 2.74	C
0.0 - 1.99	F

16

```
-- ANSWER
CREATE FUNCTION get_letter_grade(gpa DOUBLE)
RETURNS STRING
RETURN CASE
          WHEN gpa >= 3.5 THEN "A"
          WHEN gpa >= 2.75 AND gpa < 3.5 THEN "B"
          WHEN gpa >= 2 AND gpa < 2.75 THEN "C"
          ELSE "F"
       END
```

Let's apply the above UDF on the **students** table created in the previous lab

Fill in the below query to call the defined UDF on the **gpa** column

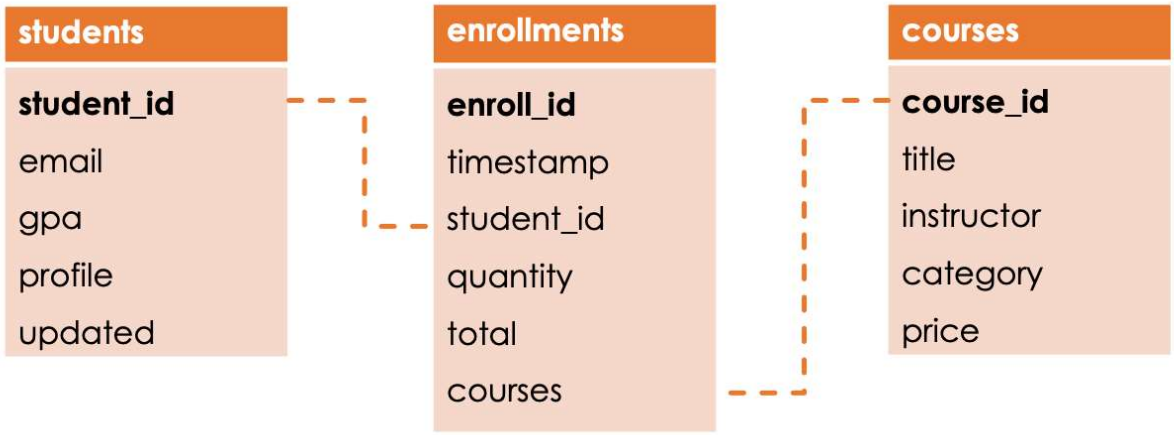
18

```
-- ANSWER
SELECT student_id, gpa, get_letter_grade(gpa) as letter_grade
FROM students
```

19

(https://databricks.com)

Lab Solution: Spark Structured Streaming



Run the following cell to setup the lab environment

5

```
%run ../Includes/Setup-Lab
```

Q1- Auto Loader

Use Auto Loader to incrementally load enrollments json files from the directory **{dataset_school}/enrollments-json-raw** into a streaming view called `enrollments_tmp_vw`

7

```
dataset_source = f"{dataset_school}/enrollments-json-raw"
schema_location = "dbfs:/mnt/DE-Associate/checkpoints/school/enrollments_stats"

# ANSWER
(spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "json")
      .option("cloudFiles.schemaLocation", schema_location)
      .load(dataset_source)
      .createOrReplaceTempView("enrollments_tmp_vw"))
```

Q2 - Calculating aggregations on streaming data

Using CTAS syntax, define a new streaming view against `enrollments_tmp_vw` to count the number of enrollments per `student_id`. Name the aggregated field: `enrollments_counts`

9

```
%sql
-- ANSWER
CREATE OR REPLACE TEMPORARY VIEW enrollments_per_student_tmp_vw AS
SELECT
  student_id, count(enroll_id) AS enrollments_count
  FROM enrollments_tmp_vw
  GROUP BY student_id
```

Q3 - Writing stream data

Stream the aggregated data from the `enrollments_per_student_tmp_vw` view to a Delta table called `enrollments_stats`.

```
11
checkpoint_path = "dbfs:/mnt/DE-Associate/checkpoints/school/enrollments_stats"

# ANSWER
query = (spark.table("enrollments_per_student_tmp_vw")
              .writeStream
              .option("checkpointLocation", checkpoint_path)
              .outputMode("complete")
              .table("enrollments_stats")
        )
```

Query the data in the `enrollments_stats` table to ensure data was written as expected.

```
13
%sql
SELECT * FROM enrollments_stats
```

Run the below cell to land a new json file of enrollments data

```
15
load_new_json_data()
```

Verify that the statistics have been updated in the table **enrollments_stats**

```
17
%sql
SELECT * FROM enrollments_stats
```

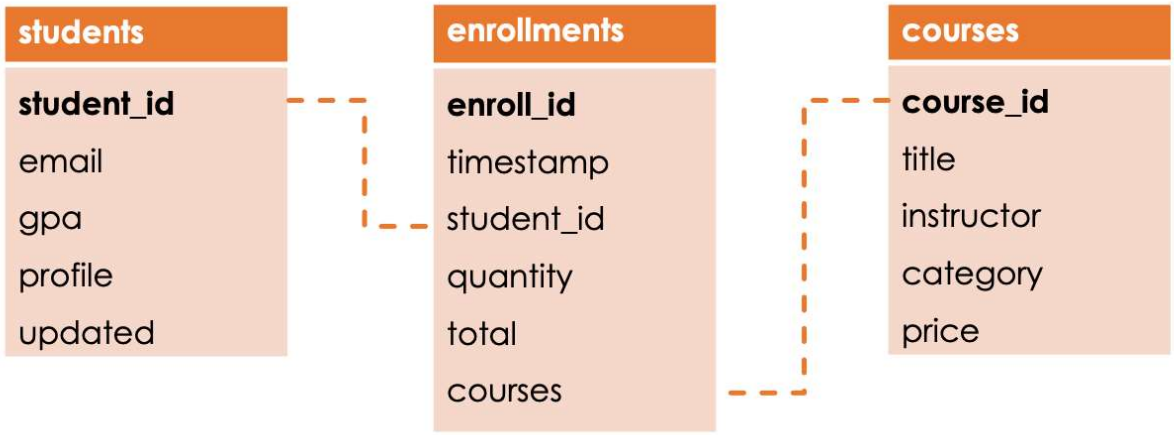
Q4 - Canceling streaming query

Finally, cancel the above streaming query

```
19
```

(https://databricks.com)

Lab Solution: Multi-Hop Architecture



Run the following cell to setup the lab environment

```
%run ../Includes/Setup-Lab
```

Q1- Declaring Bronze Table

Use Auto Loader to incrementally load enrollments json files from the directory **{dataset_school}/enrollments-json-raw** to a Delta table called `bronze`

```
dataset_source = f"{dataset_school}/enrollments-json-raw"
bronze_checkpoint_path = "dbfs:/mnt/DE-Associate/checkpoints/school/bronze"
schema_location = bronze_checkpoint_path

# ANSWER
(spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", schema_location)
        .load(dataset_source)
      .writeStream
        .option("checkpointLocation", bronze_checkpoint_path)
        .outputMode("append")
        .table("bronze")
)
```

Let's create a streaming temporary view from the bronze table in order to perform transformations using SQL.

```
9

(spark
  .readStream
  .table("bronze")
  .createOrReplaceTempView("bronze_tmp"))
```

Q2 - Data Cleansing & Enrichment

Using CTAS syntax, define a new streaming view `bronze_cleaned_tmp` against `bronze_tmp` that does the following:

- Remove records with **quantity** of 0 item
- Add a column called `processing_time` containing the current timestamp using the **current_timestamp()** function

```
11

%sql
-- ANSWER
CREATE OR REPLACE TEMPORARY VIEW bronze_cleaned_tmp AS
SELECT
  *, current_timestamp() processing_time
  FROM bronze_tmp
  WHERE quantity > 0
```

Q3 - Declaring Silver Table

Stream the data from `bronze_cleaned_tmp` to a table called `silver`.

```
13

silver_checkpoint_path = "dbfs:/mnt/DE-Associate/checkpoints/school/silver"

# ANSWER
(spark.table("bronze_cleaned_tmp")
              .writeStream
              .option("checkpointLocation", silver_checkpoint_path)
              .outputMode("append")
              .table("silver")
)
```

Let's create a streaming temporary view from the silver table in order to perform business-level aggregation using SQL

```
15
(spark
  .readStream
  .table("silver")
  .createOrReplaceTempView("silver_tmp"))
```

Q4- Declaring Gold Table

Using CTAS syntax, define a new streaming view `enrollments_per_student_tmp_vw` against `silver_tmp` to count the number of enrollments per `student`. Name the aggregated field: `enrollments_count`

```
17
%sql
-- ANSWER
CREATE OR REPLACE TEMPORARY VIEW enrollments_per_student_tmp_vw AS
SELECT
  student_id, count(enroll_id) AS enrollments_count
  FROM silver_tmp
  GROUP BY student_id
```

Stream the aggregated data from the `enrollments_per_student_tmp_vw` view to a Delta table called `gold_enrollments_stats`.

```
19
gold_checkpoint_path = "dbfs:/mnt/DE-Associate/checkpoints/school/gold_enrollments_stats"

# ANSWER
query = (spark.table("enrollments_per_student_tmp_vw")
              .writeStream
              .option("checkpointLocation", gold_checkpoint_path)
              .outputMode("complete")
              .table("gold_enrollments_stats"))
```

Query the data in the `gold_enrollments_stats` table to ensure data was written as expected.

```
21
%sql
SELECT * FROM gold_enrollments_stats
```

Run the below cell to land new a json file of enrollments data

```
23
load_new_json_data()
```

Wait for the new data to be propagated, and then run the below query to verify that the statistics have been updated in the table **gold_enrollments_stats**

```
25
%sql
SELECT * FROM gold_enrollments_stats
```

Finally, run the below cell for canceling the above streaming queries

27

```
for s in spark.streams.active:
    print("Stopping stream: " + s.id)
    s.stop()
    s.awaitTermination()
```

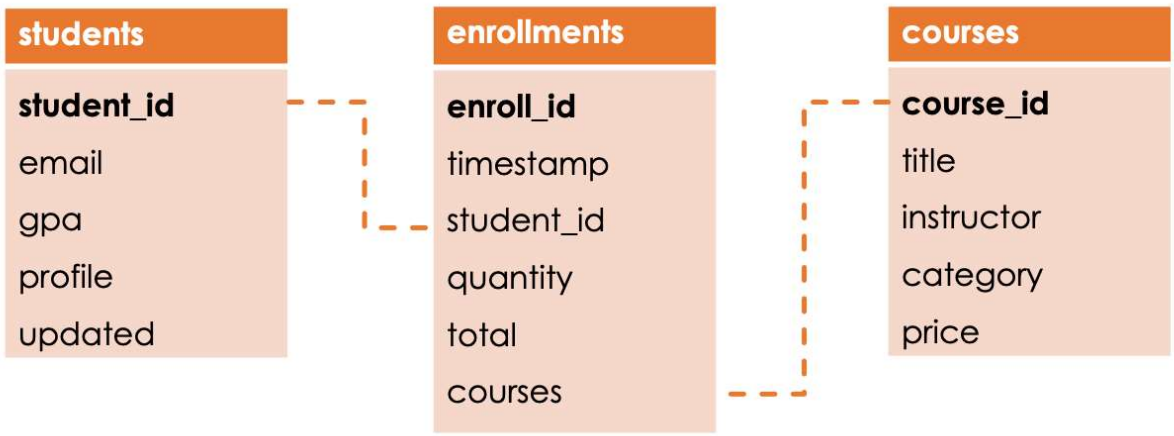
28

(https://databricks.com)

Lab Solution: implementing a DLT pipeline

This notebook is **not intended** to be executed interactively, but rather to be deployed as a DLT pipeline from the **workflows** tab

- Help: DLT syntax documentation (https://docs.databricks.com/en/delta-live-tables/tutorial-sql.html).



4

```
SET datasets.path=dbfs:/mnt/DE-Associate/datasets/school;
```

Q1- Declaring Bronze Tables

Declare a streaming live table, `enrollments_bronze`, that ingests JSON data incrementally using Auto Loader from the directory **"${datasets.path}/enrollments-json-raw"**

6

```
-- ANSWER
CREATE OR REFRESH STREAMING LIVE TABLE enrollments_bronze
AS SELECT * FROM cloud_files("${datasets.path}/enrollments-json-raw", "json",
                             map("cloudFiles.inferColumnTypes", "true"))
```

Declare a live table, `students_bronze`, that load data directly from JSON files in the directory **"${datasets.path}/students-json"**

8

```
-- ANSWER
CREATE OR REFRESH LIVE TABLE students_bronze
AS SELECT * FROM json.`${datasets.path}/students-json`
```

Q2 - Declaring Silver Table

Declare a streaming live table, `enrollments_cleaned`, that:

1. Enrich the **enrollments_bronze** data through an inner join with the `students_bronze` table on the common `student_id` field to obtain the student's country
2. Implement quality control by applying a constraint to drop records with a null `email`
3. The table will have the following schema:

Field	Type
`enroll_id`	`STRING`
`total`	`DOUBLE`
`email`	`STRING`
`country`	`STRING`

10

```
-- ANSWER
CREATE OR REFRESH STREAMING LIVE TABLE enrollments_cleaned (
  CONSTRAINT valid_email EXPECT (email IS NOT NULL) ON VIOLATION DROP ROW
)
AS SELECT enroll_id, total, email, profile:address:country as country
  FROM STREAM(LIVE.enrollments_bronze) n
  LEFT JOIN LIVE.students_bronze s
    ON n.student_id = s.student_id
```

Q3- Declaring Gold Table

Declare a live table, `course_sales_per_country` against `enrollments_cleaned` that calculate per `country` the following:

- `enrollments_count`: the number of enrollments
- `enrollments_amount`: the sum of the total amount of enrollments

Add a comment to the table: "Course Sales Per Country"

12

```
-- ANSWER
CREATE OR REFRESH LIVE TABLE course_sales_per_country
COMMENT "Course Sales Per Country"
AS
  SELECT country, count(enroll_id) AS enrollments_count, sum(total) AS enrollments_amount
  FROM LIVE.enrollments_cleaned
  GROUP BY country
```

Q4- Deploying DLT pipeline

From the **Workflows** button on the sidebar, under the **Delta Live Tables** tab, click **Create Pipeline**

Configure the pipeline settings specified below:

Setting	Instructions
Pipeline name	School DLT
Product edition	Choose **Advanced**
Pipeline mode	Choose **Triggered**
Source code	Use the navigator to select this current notebook (4.1L - Delta Live Tables)
Storage location	dbfs:/mnt/DE-Associate/dlt/school
Target schema	DE_Associate_School_DLT
Cluster policy	Leave it **None**
Cluster mode	Choose **Fixed size**
Workers	Enter **0**
Photon Acceleration	Leave it unchecked
Advanced Configuration	Click **Add Configuration** and enter: - Key: **datasets.path** - Value: **dbfs:/mnt/DE-Associate/datasets/school**
Channel	Choose **Current**

Finally, click **Create**.

Q5 - Run your Pipeline

Select **Development** mode and Click **Start** to begin the update to your pipeline's tables

Lab Solution: Creating a multi-task job

In this lab, we will create a job that has 2 tasks:

1. The current notebook that lands a new batch of data in the lab dataset directory
2. The Delta Live Table pipeline created in the previous lab to processes this data

- Help: Databricks Jobs documentation (https://docs.databricks.com/workflows/jobs/create-run-jobs.html).



Q1- Configuring Task 1 - Land New Data

From the **Workflows** button on the sidebar, under the **Jobs** tab, click the **Create Job** button.

1. Set the job name in the top-left of the screen to **School Job**
2. Configure the first task as specified below:

Setting	Value
Task name	Enter **Land New Data**
Type	Choose **Notebook**
Source	Choose **Workspace**
Path	Use the navigator to choose the current notebook (4.2L - Jobs - Land New Data)
Cluster	Select your cluster from the dropdown, under **Existing All Purpose Clusters**

3. Click the **Create** button

Q2- Configuring Task 2 - DLT pipeline

1. Click the add button (**+**) to add a new **Delta Live Tables pipeline** task
2. Configure the task:

Setting	Value
Task name	Enter **DLT pipeline**
Type	Choose **Delta Live Tables pipeline**

Setting	Value
Pipeline	Choose the DLT pipeline created in the previous lab
Depends on	Choose **Land New Data**, which is the previous task we defined above

Q3- Run the job

Click the **Run now** button in the top right to run this job. From the **Runs** tab, check your job run

Q4- Review the finished job

Once all tasks completed successfully, review the contents of each task to verify its result

> **Note**: The below cells are to be run as part of the **Task 1** to land new batch of data in the dataset directory

```
9
%run ../Includes/Setup-Lab
```

```
10
load_new_json_data()
```

```
11
```

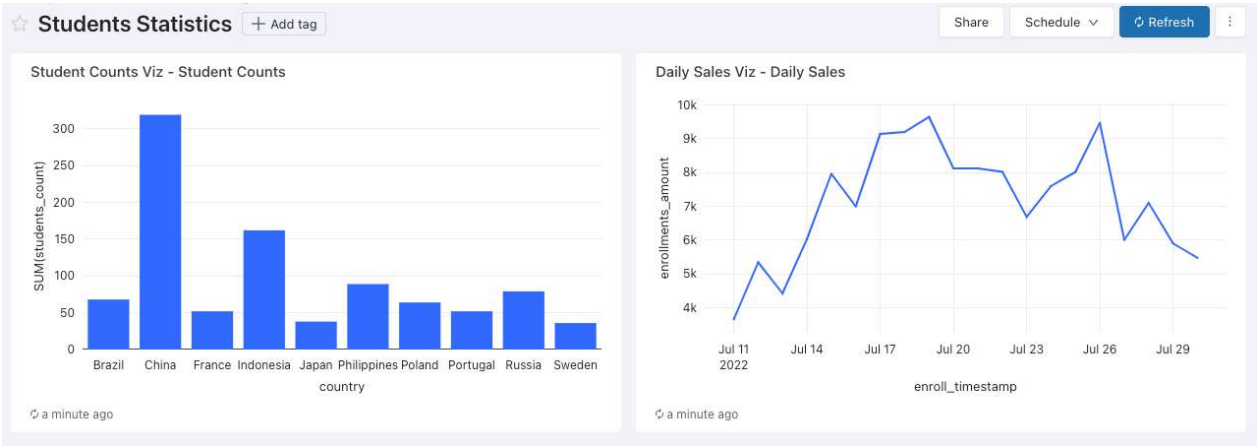
(https://databricks.com)

Lab Solution: Design a Dashboard with DBSQL

In this lab, we will design a dashboard in DBSQL that has 2 graphs:

1. Bar graph that shows the number of students per country
2. Line graph that shows the daily enrollments amount

- Help: Databricks SQL documentation (https://docs.databricks.com/sql/index.html#what-is-databricks-sql).



Q1 - Working with queries in SQL Editor

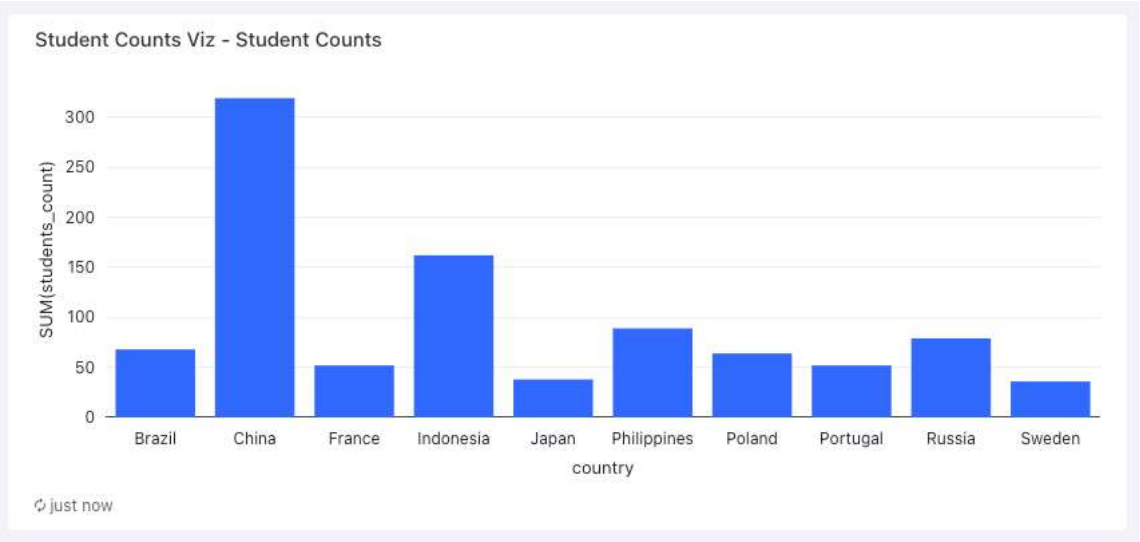
Run the the below query in **SQL Editor** in Databricks SQL, and then save it with the name **Student Counts**

5

```
SELECT profile:address:country as country, count(student_id) AS students_count
FROM hive_metastore.de_associate_school.students
GROUP BY profile:address:country
ORDER BY students_count DESC
LIMIT 10
```

Q2 - Creating a Bar Graph Visualization

Create a bar graph that shows the number of students per country



Anwser:

Steps:

Q3 - Creating a New Dashboard

Add the above graph to a new dashboard named **Students Statistics**

Anwser:

Steps:

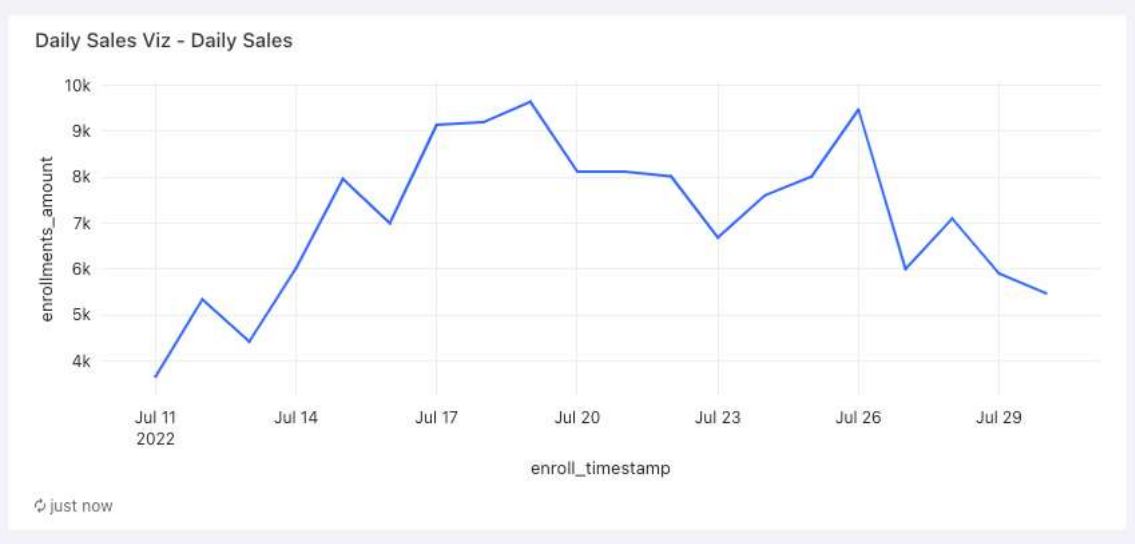
1. Click the three vertical dots button at the top of the graph and select **Add to Dashboard**.
2. Click the **Create new dashboard** option
3. Name your dashboard **Students Statistics**
4. Click **Save**
5. With the new dashboard selected as the target, click **OK** to add your visualization

Q4 - Creating a Line Plot Visualization

1. Run the the below query in a new query tab in the **SQL Editor**, and then save it with the name **Daily Sales**


```
SELECT cast(from_unixtime(enroll_timestamp, 'yyyy-MM-dd HH:mm:ss') AS date) enroll_timestamp,
       sum(total) AS enrollments_amount
FROM hive_metastore.de_associate_school.enrollments n
INNER JOIN hive_metastore.de_associate_school.students s ON s.student_id = n.student_id
GROUP BY enroll_timestamp
```

2. Create a Line Plot Visualization that shows the daily enrollments amount



Anwser:

Steps:

1. Click the **Add Visualization** button
2. Select `Line` for the **Visualization Type**
3. Set `enroll timestamp` for the **X Column**

Q5 - Review your Dashboard

Open your Dashboard and refresh its underlaying data

Anwser:

Steps:

1. Click on the **Dashboards** button on left side bar
2. Find the dashboard **Students Statistics** created earlier. Click to open it
3. Click the **Refresh** button to update your dashboard

databricks Includes / Setup-Lab

(https://databricks.com)

2

```
data_source_uri = "wasbs://course-resources@dalhussein.blob.core.windows.net/datasets/school/v1/"
dataset_school = 'dbfs:/mnt/DE-Associate/datasets/school'
checkpoint_path = 'dbfs:/mnt/DE-Associate/checkpoints/school'
dlt_path = 'dbfs:/mnt/DE-Associate/dlt/school'
db_name = 'DE_Associate_School'
dlt_db_name = 'DE_Associate_School_DLT'
spark.conf.set(f"dataset.school", dataset_school)
```

3

```
def clean_up():
    print("Removing Checkpoints ...")
    dbutils.fs.rm(checkpoint_path, True)
    print("Removing DLT storage location ...")
    dbutils.fs.rm(dlt_path, True)
    print("Dropping Database ...")
    spark.sql(f"DROP SCHEMA IF EXISTS {db_name} CASCADE")
    print("Dropping DLT database ...")
    spark.sql(f"DROP SCHEMA IF EXISTS {dlt_db_name} CASCADE")
    print("Removing Dataset ...")
    dbutils.fs.rm(dataset_school, True)
    print("Done")
```

4

```
try:
    clean = int(dbutils.widgets.get("clean"))
except:
    clean = 0

if clean:
    clean_up()
```

5

```
def path_exists(path):
  try:
    dbutils.fs.ls(path)
    return True
  except Exception as e:
    if 'java.io.FileNotFoundException' in str(e):
      return False
    else:
      raise
```

6

```
def download_dataset(source, target):
    files = dbutils.fs.ls(source)

    for f in files:
        source_path = f"{source}/{f.name}"
        target_path = f"{target}/{f.name}"
        if not path_exists(target_path):
            print(f"Copying {f.name} ...")
            dbutils.fs.cp(source_path, target_path, True)
```

7

```
def get_index(dir):
    files = dbutils.fs.ls(dir)
    index = 0
    if files:
        file = max(files).name
        index = int(file.rsplit('.', maxsplit=1)[0])
    return index+1
```

8

```
def set_current_schema(schema_name, catalog_name='hive_metastore'):
    spark.sql(f"USE CATALOG {catalog_name}")
    spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema_name}")
    spark.sql(f"USE {schema_name}")
    print(f"Schema for the hands-on labs: {catalog_name}.{schema_name}")
```

9

```
# Structured Streaming
streaming_dir = f"{dataset_school}/enrollments-streaming"
raw_dir = f"{dataset_school}/enrollments-raw"

def load_file(current_index):
    latest_file = f"{str(current_index).zfill(2)}.parquet"
    print(f"Loading {latest_file} file to the school dataset")
    dbutils.fs.cp(f"{streaming_dir}/{latest_file}", f"{raw_dir}/{latest_file}")


def load_new_data(all=False):
    index = get_index(raw_dir)
    if index >= 10:
        print("No more data to load\n")

    elif all == True:
        while index <= 10:
            load_file(index)
            index += 1
    else:
        load_file(index)
        index += 1
```

10

```
# DLT
streaming_enrollments_dir = f"{dataset_school}/enrollments-json-streaming"
streaming_courses_dir = f"{dataset_school}/courses-streaming"

raw_enrollments_dir = f"{dataset_school}/enrollments-json-raw"
raw_courses_dir = f"{dataset_school}/courses-cdc"

def load_json_file(current_index):
    latest_file = f"{str(current_index).zfill(2)}.json"
    print(f"Loading {latest_file} enrollments file to the school dataset")
    dbutils.fs.cp(f"{streaming_enrollments_dir}/{latest_file}", f"{raw_enrollments_dir}/{latest_file}")
    #print(f"Loading {latest_file} courses file to the school dataset")
    #dbutils.fs.cp(f"{streaming_courses_dir}/{latest_file}", f"{raw_courses_dir}/{latest_file}")


def load_new_json_data(all=False):
    index = get_index(raw_enrollments_dir)
    if index >= 10:
        print("No more data to load\n")

    elif all == True:
        while index <= 10:
            load_json_file(index)
            index += 1
    else:
        load_json_file(index)
        index += 1
```

11

```
download_dataset(data_source_uri, dataset_school)
set_current_schema(db_name)
```

12